Universidad Tecmilenio	**Gestión de proyectos de inteligencia artificial**
	Docente: Tania Judith Mendoza Torres
	AL02568360 Ricardo Alejandro Lastra Dueñas
	Fase II. Ejecución ❖ Documentación técnica de despliegue

1. Información general

Componente	Descripción
Nombre del proyecto	WODPredict-AI: sistema inteligente de predicción de sobrecarga al día siguiente en atletas de CrossFit
Versión del documento	1.0
Fecha	1 de noviembre de 2026
Responsables técnicos	Ricardo Alejandro Lastra Dueñas (ciencia de datos, desarrollo y despliegue)
Organización / institución	UnderCover Fitness, red de centros de CrossFit (caso de estudio)
Estado del proyecto	Prototipo validado: contenerizado y probado en entorno local y desplegado en Azure Container Apps (Azure for Students, región México Central); piloto con datos reales planeado para noviembre de 2026
Repositorio	https://github.com/rick-lastra/WODPredict-AI_etapa-4_API-despliegue

2. Introducción

2.1 Propósito del documento

Este documento describe la arquitectura, el proceso de despliegue, los componentes operativos y las consideraciones técnicas del sistema WODPredict-AI, con base en la norma ISO/IEC 23053:2022 (marco para sistemas de inteligencia artificial que usan aprendizaje automático). La norma es un marco conceptual, no prescriptivo; por ello este documento traduce sus componentes, funciones y terminología a la documentación del despliegue, manteniendo coherencia normativa y trazabilidad. Se elaboró con apoyo de IA generativa. Está escrito para que otra persona pueda construir, ejecutar, probar y desplegar la solución siguiendo los pasos, sin depender de la computadora del autor.

2.2 Alcance

El alcance de este documento comprende el sistema de IA en fase de operación y despliegue, incluidos sus componentes de datos, modelo y servicio de inferencia; las interacciones entre componentes (cliente, API, modelo e infraestructura); la contenerización con Docker y el despliegue local y en Azure Container Apps, así como las consideraciones técnicas, éticas y operativas de monitoreo y mantenimiento. Quedan fuera del alcance el análisis exploratorio y el entrenamiento avanzado, documentados en las Etapas 2 y 3; la optimización profunda de infraestructura, la orquestación con Kubernetes, la autenticación de usuarios y la integración con Wodify, planeadas para la Fase III.

3. Marco normativo y referencias

El despliegue de WODPredict-AI se sustenta en un marco normativo que combina estándares internacionales de inteligencia artificial y la legislación mexicana en materia de protección de datos. La norma ISO/IEC 23053:2022 (Framework for Artificial Intelligence Systems Using Machine Learning) define los componentes y funciones de un sistema de IA basado en aprendizaje automático y sirve como estructura de este documento; la ISO/IEC 22989:2022 (Artificial intelligence concepts and terminology) aporta la terminología empleada en el glosario, y la ISO/IEC 23894:2023 (Artificial intelligence: Guidance on risk management) orienta la identificación y mitigación de riesgos descrita en la sección 10. En el ámbito legal, la Ley Federal de Protección de Datos Personales en Posesión de los Particulares establece los lineamientos para el tratamiento de los datos de los atletas durante el piloto con datos reales. Como referencia complementaria, se retoma la

documentación interna del proyecto: el Business Case de la Fase I, los reportes técnicos de las Etapas 2 y 3, el README del repositorio y el Documento de validación y pruebas.

4. Visión general del sistema de IA

4.1 Objetivo del sistema de IA

Problema. Las lesiones por acumulación de fatiga son la principal causa de deserción de atletas en UnderCover Fitness y hoy el coach solo cuenta con la regla ACWR > 1.5, que detecta 3 de cada 10 sobrecargas. **Objetivo.** Anticipar si un atleta tendrá sobrecarga al día siguiente a partir de información que se conoce antes de la sesión (carga y frecuencia de los últimos días, ACWR, RPE de la sesión anterior y probabilidad de fatiga de sus comentarios), para que el coach ajuste la carga. **Valor.** Detectar 29 de cada 39 sobrecargas por cada 100 atletas a la semana (contra 12 con la regla actual) y contribuir a reducir en un 25 % la deserción.

4.2 Tipo de sistema de IA

Sistema de IA basado en machine learning supervisado, con un modelo de clasificación binaria: 1 = sobrecarga al día siguiente, 0 = sin sobrecarga. La salida es una probabilidad que se compara con un umbral de 0.17 y se traduce en tres niveles de riesgo para el coach (bajo, moderado y alto).

4.3 Actores involucrados

Actor	Rol	Interacción con el sistema
Coaches	Usuarios finales	Consultan el riesgo en el formulario web y deciden el ajuste de carga
Head coach / analista de datos	Operador del sistema	Revisa alertas, métricas y el desempeño del modelo
Responsable técnico (TI)	Administrador técnico	Construye la imagen, despliega en Azure, monitorea y actualiza versiones
Atletas	Sujetos de los datos	Sus registros alimentan el modelo; no interactúan con el sistema

5. Arquitectura del sistema de IA

5.1 Descripción general de la arquitectura

La arquitectura es modular y relaciona tres grupos de componentes: de datos (entrada del atleta, validación y preprocesamiento), de ML (modelo entrenado y motor de inferencia) y de operación y soporte (contenedor, health check, logs, pruebas y plataforma en la nube). Vista lógica: el modelo está desacoplado de la interfaz y se consulta mediante un servicio REST de inferencia. El cliente (formulario web, Swagger o cualquier sistema) envía los datos del atleta en JSON; la API valida el contrato de entrada y llama al modelo; el resultado regresa en JSON. Docker empaqueta API, modelo, frontend y dependencias en una sola imagen que se ejecuta igual en la laptop y en Azure.

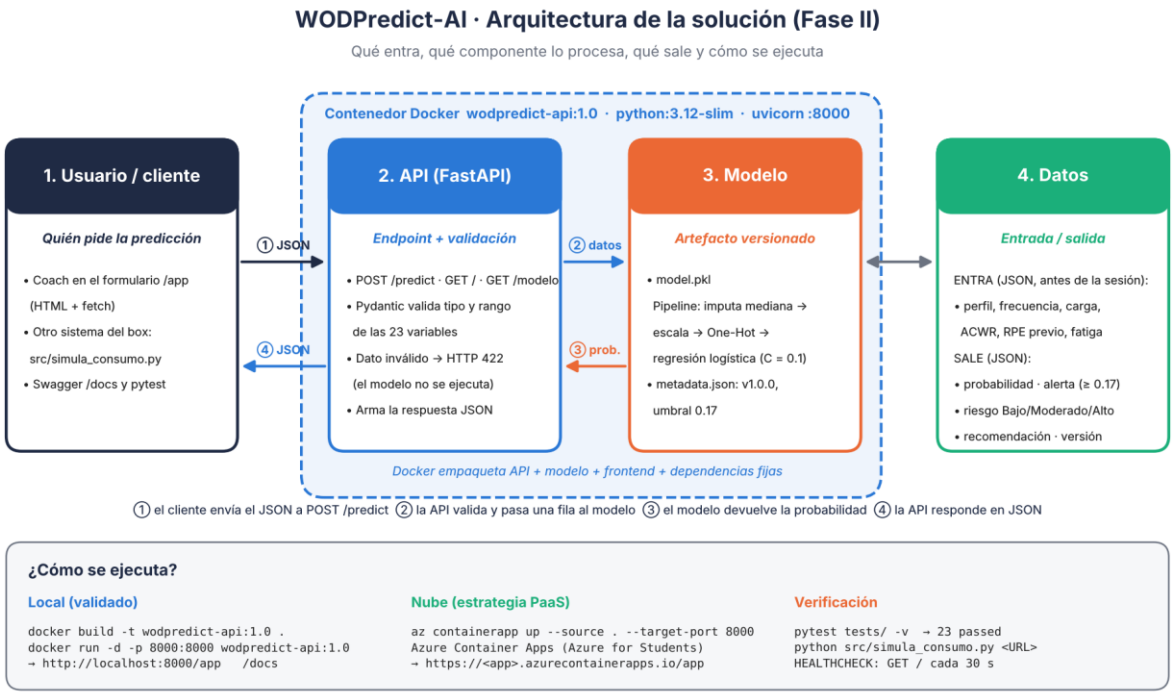


Figura 1. Arquitectura de WODPredict-AI: qué entra, qué componente procesa, qué sale y cómo se ejecuta (Anexo A).

En desarrollo, un contenedor Docker (Windows con Docker Desktop, puerto 8000). En operación, una réplica del mismo contenedor en Azure Container Apps, con imagen guardada en Azure Container Registry, acceso público por HTTPS y logs en Log Analytics.

5.2 Componentes del sistema (alineados a ISO/IEC 23053)

Componente ISO/IEC 23053	Implementación en WODPredict-AI	Archivo / recurso
Fuente de datos	Dataset sintético de sesiones y atletas (rick-lastra/wodpredict-synth@v1.0); en el piloto, exportaciones de Wodify	Hugging Face / Etapa 2
Procesamiento de datos	Limpieza y estandarización (Etapa 2); en inferencia: validación de tipos y rangos, imputación con la mediana, escalado y One-Hot dentro del Pipeline	src/app.py, model/
Modelo ML	Regresión logística (C = 0.1) entrenada offline con src/entrenar_modelo.py, umbral 0.17; se carga con src/model_loader.py	model/model.pkl, model/metadata.json
Motor de inferencia	Servicio REST con FastAPI y uvicorn (POST /predict)	src/app.py
Interfaz / API	API REST (/predict, /modelo, /docs) y formulario web que consume la API; Swagger para pruebas técnicas	frontend/index.html, /docs
Infraestructura	Contenedor Docker (python:3.12-slim) en local y en Azure Container Apps	Dockerfile
Herramientas de verificación	pytest, tabla de casos extremos, cliente externo con requests y notebook de evidencias	tests/, src/simula_consumo.py, notebooks/

6. Datos en el entorno de despliegue

6.1 Tipos de datos

Datos de entrada (23 variables que se conocen antes de la sesión):

Grupo	Variables	Ejemplo
Perfil del atleta	level, sex, language, age, days_since_first_session	élite, F, es, 31 años
Frecuencia	days_since_prev_session, sessions_last_3d, sessions_last_7d, sessions_last_28d	5 sesiones en 7 días
Carga (RPE × minutos)	load_last_3d, load_last_7d, mean_session_load_28d, acwr3_freq, acwr3_load, acwr_freq, acwr_load	ACWR de carga 1.25

Sesión anterior	prev_rpe, prev_session_load, prev_has_note	RPE 8, carga 420
Comentarios	prob_fatiga_prev_session, prob_fatiga_last_note, days_since_last_note, notes_last_28d	probabilidad de fatiga 0.70

Datos de salida: probabilidad de sobrecarga (0 a 1), alerta (1 si la probabilidad ≥ 0.17), nivel de riesgo (Bajo, Moderado o Alto), recomendación para el coach y versión del modelo.

Datos derivados: las variables de carga y ACWR se calculan a partir del historial de sesiones (RPE × minutos); dentro del pipeline se generan la imputación de faltantes, el escalado y las columnas One-Hot de nivel, sexo e idioma.

6.2 Gestión de datos

- **Validación al ingreso:** cada variable tiene tipo y rango (por ejemplo, RPE de 1 a 10 y probabilidades de 0 a 1). Un dato inválido recibe HTTP 422 y no llega al modelo.
- **Datos faltantes:** edad, RPE, ACWR y variables de comentarios son opcionales; el pipeline los imputa con la mediana del entrenamiento.
- **Almacenamiento:** la API no almacena las peticiones ni las respuestas; solo procesa y responde. El modelo y su metadata viven dentro de la imagen.
- **Protección de datos:** la API no recibe nombre ni identificador del atleta (minimización); el usuario del contenedor no tiene privilegios de administrador.
- **Periodos de retención:** las peticiones no se retienen; los logs de Azure (solo método, ruta, código y tiempo) se conservan 30 días en Log Analytics; los datos de entrenamiento se conservan versionados mientras el modelo esté en uso.
- **Transporte seguro:** en Azure el acceso es por HTTPS con certificado administrado por la plataforma.
- **Datos de entrenamiento:** sintéticos y versionados (tag v1.0); no contienen información personal real.

7. Modelo de machine learning en producción

7.1 Descripción del modelo

Elemento	Descripción
Algoritmo	Regresión logística (C = 0.1) en un Pipeline de scikit-learn 1.6.1, seleccionada en las Etapas 2 y 3 por ser la más simple e interpretable sin diferencia significativa frente a Random Forest y XGBoost
Variables de entrada	23 variables conocidas antes de la sesión (sección 6.1)
Variables de salida	Probabilidad de sobrecarga, alerta (probabilidad ≥ 0.17), nivel de riesgo y recomendación para el coach
Métricas del modelo	Recall 0.732 (métrica principal, meta ≥ 0.70), precisión 0.378, F1 0.498 y ROC-AUC 0.859 en 16,498 sesiones de 56 atletas nuevos
Métricas en fase de despliegue	Latencia de /predict: mediana 17.3 ms y p95 22.4 ms; mismas probabilidades antes y después de Docker; 23 de 23 pruebas aprobadas
Artefacto	model/model.pkl (6 KB) + model/metadata.json (versión, umbral, variables, métricas y versiones de librerías)

7.2 Ciclo de vida del modelo

Siguiendo ISO/IEC 23053, el entrenamiento y la operación están separados: el modelo se entrena offline con src/entrenar_modelo.py, y en operación la API solo carga el artefacto y hace inferencia; nunca reentrena con los datos que recibe. Versión actual: 1.0.0.

1. **Entrenamiento offline** con src/entrenar_modelo.py sobre los datos versionados (separación 80/20 por atleta).
2. **Validación previa al despliegue:** métricas en prueba y umbral elegido con validación cruzada (Etapa 3); pruebas del sistema en contenedor (Documento de validación y pruebas).
3. **Despliegue:** el artefacto se copia a la imagen Docker; la API solo lo carga, nunca reentrena.

4. **Monitoreo:** alertas, latencia y desempeño contra los reportes del coach (sección 9).
5. **Estrategia de actualización:** reentrenamiento mensual durante el piloto con los datos nuevos de Wodify; la nueva versión se publica como una nueva etiqueta de imagen.
6. **Criterios de reemplazo:** se sustituye el modelo si el recall semanal baja de 0.70, la precisión de 0.35, o si el porcentaje de alertas cambia más de 10 puntos (posible data drift); el nuevo modelo debe superar al actual en el mismo conjunto de prueba antes de desplegarse.

8. Proceso de despliegue del sistema

8.1 Estrategia de despliegue

Tipo de servicio: API REST (consulta en línea, una predicción por petición), no batch ni servicio embebido. **Entorno:** local para desarrollo y validación, y nube pública para operación. La estrategia es API REST + contenerización con Docker + PaaS. Primero se valida la API y el contenedor en local; después la misma imagen se publica en Azure Container Apps con la cuenta académica Azure for Students. Se compararon los servicios de Azure disponibles con la cuenta académica:

Servicio de Azure	Modelo	Ventajas	Desventajas	Costo con Azure for Students
Azure Container Apps (elegido)	PaaS serverless de contenedores	Un solo comando; HTTPS automático; escala a 0 sin uso; logs integrados	Arranque en frío de unos segundos tras inactividad	Capa gratuita mensual (180,000 vCPU-s, 360,000 GiB-s y 2 millones de peticiones); el registro ACR Basic consume ≈ USD 5 al mes del crédito
Azure App Service (Web App for Containers)	PaaS	Despliegue desde la extensión de Docker de VS Code	El plan B1 cobra, aunque no se use	≈ USD 13 al mes del crédito
Azure Virtual Machine	IaaS	Control total del servidor	Hay que administrar SO, Docker, red y seguridad	Cobra por hora encendida

Se elige Azure Container Apps en la región México Central (la más cercana al box entre las permitidas por la suscripción) porque se entrega la aplicación ya contenerizada y Azure administra servidores, red, certificados y escalado; para un piloto con pocos coaches el uso cabe en la capa gratuita. Es una nube pública, adecuada mientras los datos sean sintéticos; con datos reales de atletas se evaluaría un esquema híbrido (datos personales en Wodify y solo la inferencia en la nube).

8.2 Pipeline de despliegue

El despliegue se organiza en cuatro fases. Los pasos 1 a 6 corresponden a la simulación local del despliegue (modelo, API, ejecución, prueba, consumo y Docker) con el proyecto WODPredict-AI, y los pasos 7 a 10 la llevan a la nube con Azure.

Fase	Pasos	Resultado
1. Preparación del entorno	Requerimientos técnicos y paso 1	Equipo con Docker, Python, VS Code y el repositorio
2. Implementación del modelo	Pasos 2 a 6	Artefacto del modelo servido por la API dentro de un contenedor healthy
3. Configuración de servicios	Pasos 7 y 8	Código en GitHub y aplicación publicada en Azure Container Apps
4. Validación postdespliegue	Pasos 9 y 10	Pruebas aprobadas contra la URL pública y costos controlados

Fase 1. Preparación del entorno

Requerimientos técnicos:

Requisito	Versión	Para qué
Windows 10/11 con WSL 2	—	Sistema operativo del equipo de desarrollo
Docker Desktop	4.x (motor 29.x)	Construir y ejecutar la imagen
VS Code	Extensiones Python, Jupyter, Docker y GitHub	Editar, ejecutar el notebook y administrar contenedores
Python	3.12	Pruebas, cliente externo y notebook (la API corre dentro de Docker)
Git y cuenta de GitHub	2.x	Control de versiones y repositorio
Azure CLI	2.6x o superior + extensión containerapp	Despliegue en Azure
Cuenta Azure for Students	Crédito académico	Azure Container Apps y Azure Container Registry
Hardware	4 GB de RAM libres, 2 GB de disco	La imagen ocupa ≈ 690 MB en disco (≈ 160 MB comprimida)

Dependencias de la API con versión fija (requirements.txt): numpy 2.0.2, scipy 1.16.3, pandas 2.2.2, scikit-learn 1.6.1, joblib 1.6.0, fastapi 0.162.0, pydantic 2.14.2 y uvicorn 0.54.0. Para pruebas y notebook se agregan las de requirements-dev.txt (pytest, requests, httpx, matplotlib e ipykernel).

Paso 1. Preparar el entorno. Clonar el repositorio, abrirlo en VS Code y crear el ambiente de Python (terminal de VS Code, PowerShell):

```
git clone https://github.com/rick-lastra/WODPredict-AI_etapa-4_API-despliegue.git
cd WODPredict-AI_etapa-4_API-despliegue
code .
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
```

Fase 2. Implementación del modelo

Paso 2. Generar el artefacto del modelo (opcional: el repositorio ya incluye model/). Entrena la regresión logística con los datos v1.0 y guarda el model.pkl y su metadata.json:

```
python src/entrenar_modelo.py
# Métricas en prueba con umbral 0.17: recall 0.732, ROC-AUC 0.859
```

Paso 3. Ejecutar la API en local. Uvicorn levanta el servidor en http://127.0.0.1:8000 y la documentación interactiva en /docs:

```
uvicorn src.app:app --reload
```

Paso 4. Probar el sistema. En http://127.0.0.1:8000/docs abrir POST /predict > *Try it out* > *Execute* con el ejemplo precargado (riesgo moderado, probabilidad 0.3025). En http://127.0.0.1:8000/app usar el botón *Ejemplo: riesgo alto* y *Calcular riesgo* (probabilidad 0.7502, riesgo Alto).

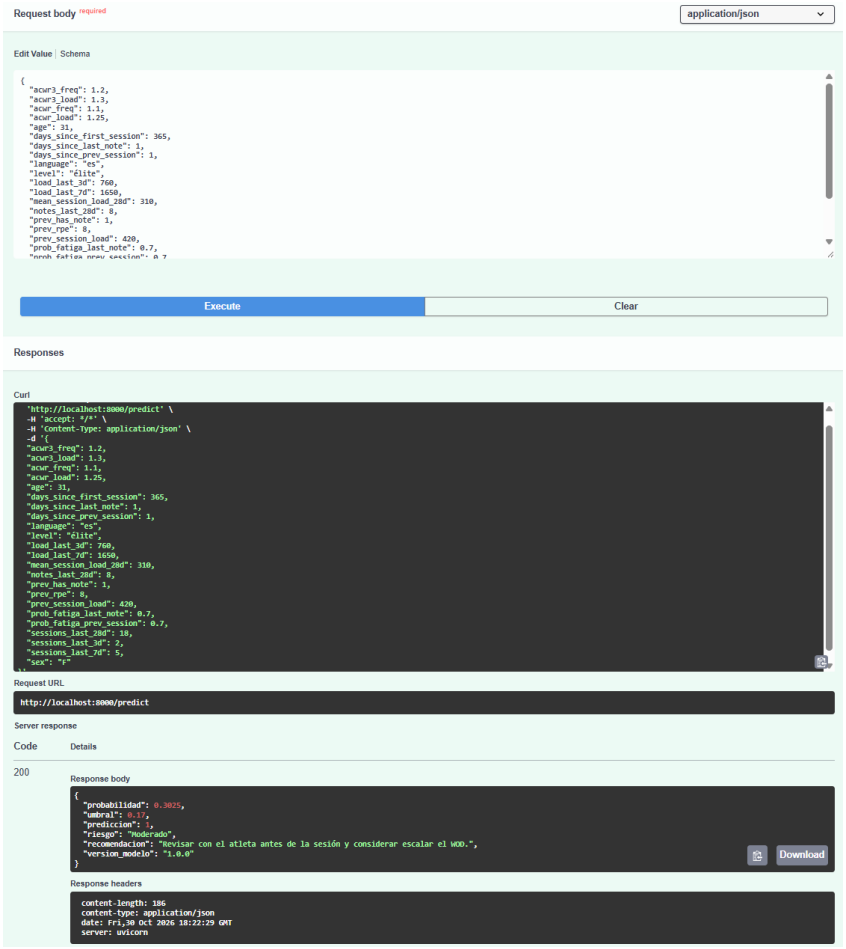


Figura 2. Swagger /docs en el contenedor local: POST /predict ejecutado con el ejemplo precargado y respuesta real (HTTP 200, probabilidad 0.3025, riesgo Moderado).

Paso 5. Simular el consumo del sistema. src/simula_consumo.py representa a otro sistema del box que consume la API con requests:

```
python src/simula_consumo.py
# Atleta de ejemplo 'alto' -> HTTP 200
#   Riesgo Alto (75.0%) | alerta: 1
# Dato inválido (prev_rpe = 11) -> HTTP 422: prev_rpe - Input should be less than or equal to 10
```

Paso 6. Contenerización con Docker. Detener uvicorn (Ctrl + C), abrir Docker Desktop y construir y ejecutar la imagen:

```
docker build -t wodpredict-api:1.0 .
docker run -d --name wodpredict -p 8000:8000 wodpredict-api:1.0
docker ps              # STATUS: Up ... (healthy)
pytest tests/ -v       # 23 passed
```

El Dockerfile (Anexo C) parte de python:3.12-slim, instala requirements.txt, copia solo src/app.py, src/model_loader.py, model/ y frontend/, crea un usuario sin privilegios, expone el puerto 8000 y define un HEALTHCHECK sobre GET /. El mismo puerto (8000) se usa en local y en Azure.

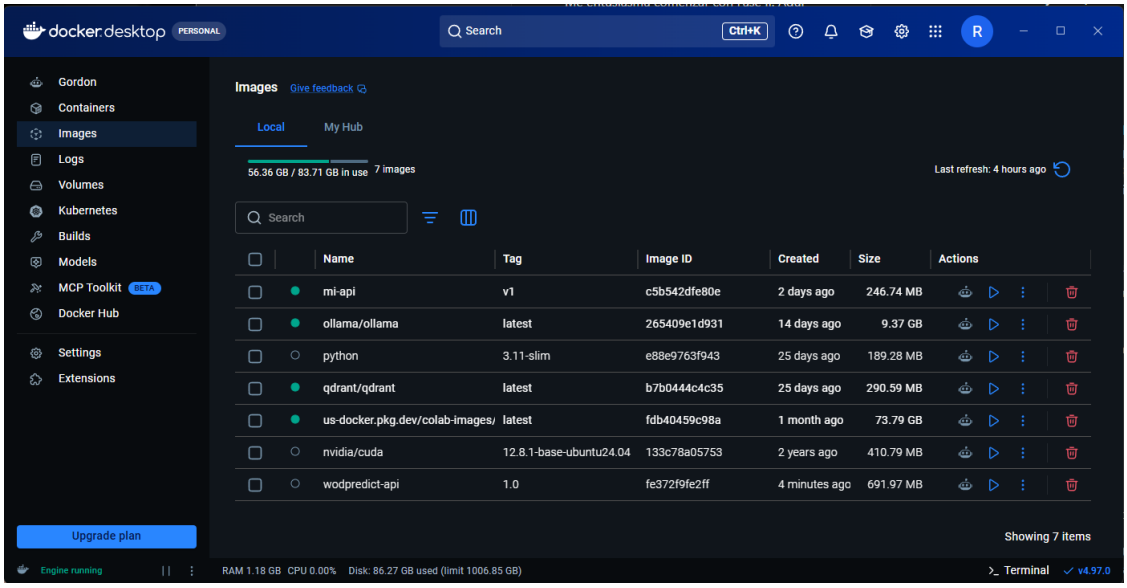


Figura 3. Docker Desktop > Images: imagen wodpredict-api:1.0 (692 MB) construida en el equipo del autor.

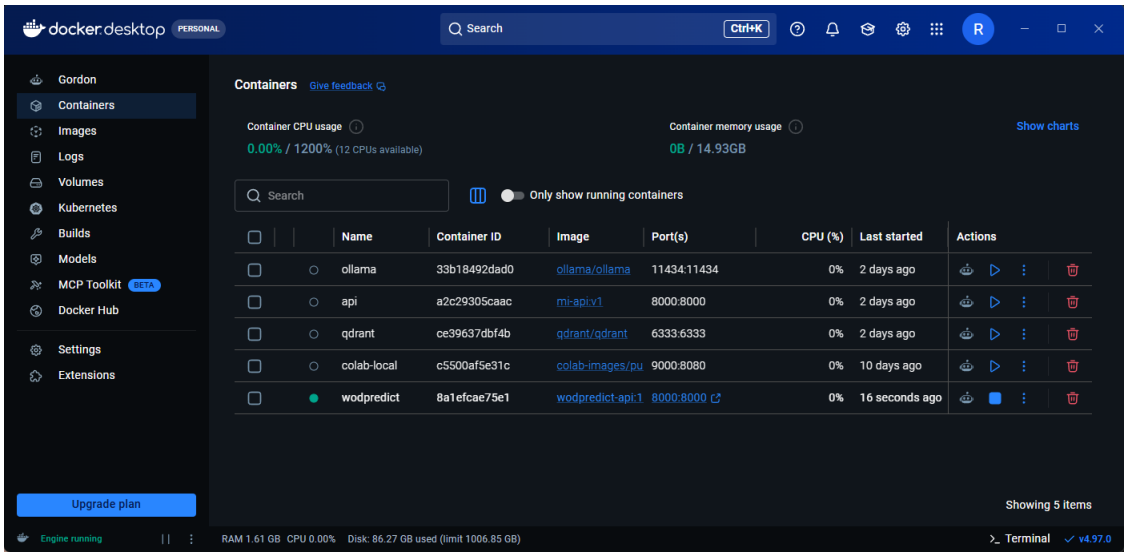


Figura 4. Docker Desktop > Containers: contenedor wodpredict en ejecución con el puerto 8000:8000 publicado.

Fase 3. Configuración de servicios

Paso 7. Control de versiones en GitHub. Una vez revisado el código, crear en GitHub el repositorio WODPredict-AI-FaseII (sin README, porque ya existe) y subir el proyecto a la rama main. El archivo .gitignore deja fuera lo que solo sirve para trabajar en local y no hace falta para clonar y replicar la solución: .venv/, cachés de Python, credenciales (.env), datos descargados, la carpeta evidencias/ que regenera el notebook y los archivos editables .docx (en docs/ solo se suben los PDF). Antes del commit, git status debe mostrar únicamente los archivos de la estructura del README:

```
git init
git add .
git status          # revisar que no aparezcan .venv, evidencias/ ni .docx
git commit -m "WODPredict-AI Fase II: API, Docker, pruebas y documentación"
git branch -M main
git remote add origin https://github.com/rick-lastra/WODPredict-AI_etapa-4_API-despliegue.git
git push -u origin main
git tag v1.0-fase2; git push origin v1.0-fase2
```

Paso 8. Despliegue en Azure Container Apps. Iniciar sesión con la cuenta académica y preparar la CLI (solo la primera vez). La suscripción Azure for Students tiene tres restricciones que definen este procedimiento: exige autenticación multifactor para administrar recursos, solo permite cinco regiones (northcentralus, mexicocentral, centralus, westus y eastus2; se consultan con az policy assignment list) y no permite ACR Tasks, por lo que la imagen no se construye en la nube. Por eso se publica en Azure Container Registry la misma imagen que se validó en local y Container Apps la ejecuta:

```
winget install -e --id Microsoft.AzureCLI
az login                        # si pide MFA, repetir con --claims-challenge
az account show --output table        # suscripción "Azure for Students"
az extension add --name containerapp --upgrade
az provider register --namespace Microsoft.App --wait
az provider register --namespace Microsoft.OperationalInsights --wait
az provider register --namespace Microsoft.ContainerRegistry --wait

# Grupo de recursos y registro de contenedores en México Central
az group create -n rg-wodpredict-mx -l Mexicocentral
az acr create -n wodpredictal02568360 -g rg-wodpredict-mx -l Mexicocentral --sku Basic --admin-enabled
true

# Publicar la imagen local validada
az acr login -n wodpredictal02568360
docker tag wodpredict-api:1.0 wodpredictal02568360.azurecr.io/wodpredict-api:1.0
docker push wodpredictal02568360.azurecr.io/wodpredict-api:1.0

# Crear el ambiente y la Container App con esa imagen
$clave = az acr credential show -n wodpredictal02568360 --query "passwords[0].value" -o tsv
az containerapp up --name wodpredict-api-mx --resource-group rg-wodpredict-mx `
 --location Mexicocentral --image wodpredictal02568360.azurecr.io/wodpredict-api:1.0 `
 --registry-server wodpredictal02568360.azurecr.io --registry-username wodpredictal02568360 `
 --registry-password $clave --ingress external --target-port 8000
```

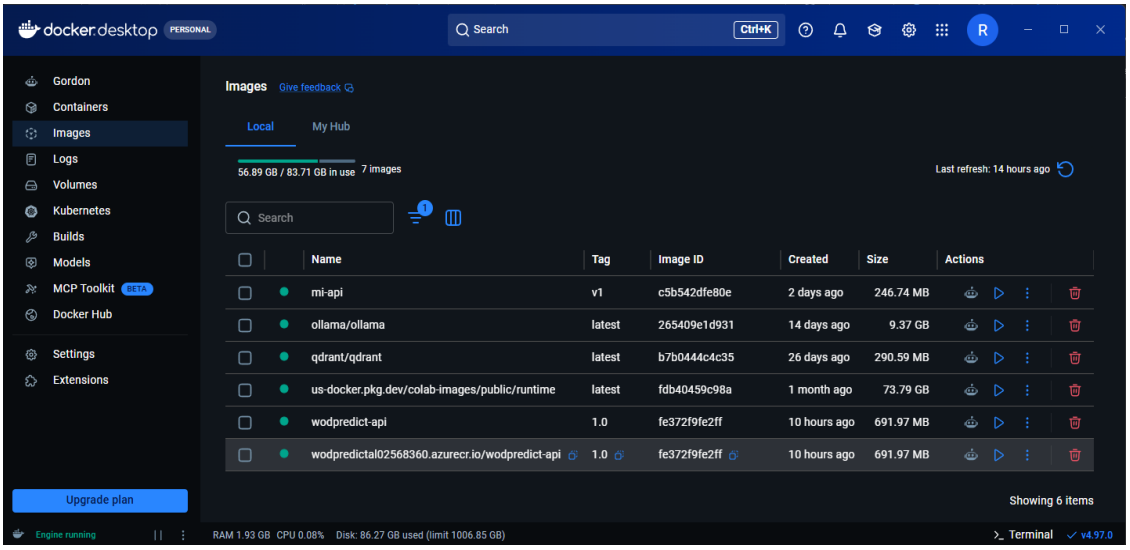


Figura 5. Docker Desktop > Images: la imagen validada en local (ID fe372f9fe2ff) etiquetada para Azure Container Registry antes del docker push; ambas etiquetas son la misma imagen.

Resultado: Azure crea el ambiente wodpredict-api-mx-env con Log Analytics y publica la aplicación en https://wodpredict-api-mx.calmcliff-e9faa155.Méxicocentral.azurecontainerapps.io. La Figura 6 muestra la Container App en estado Running en México Central.

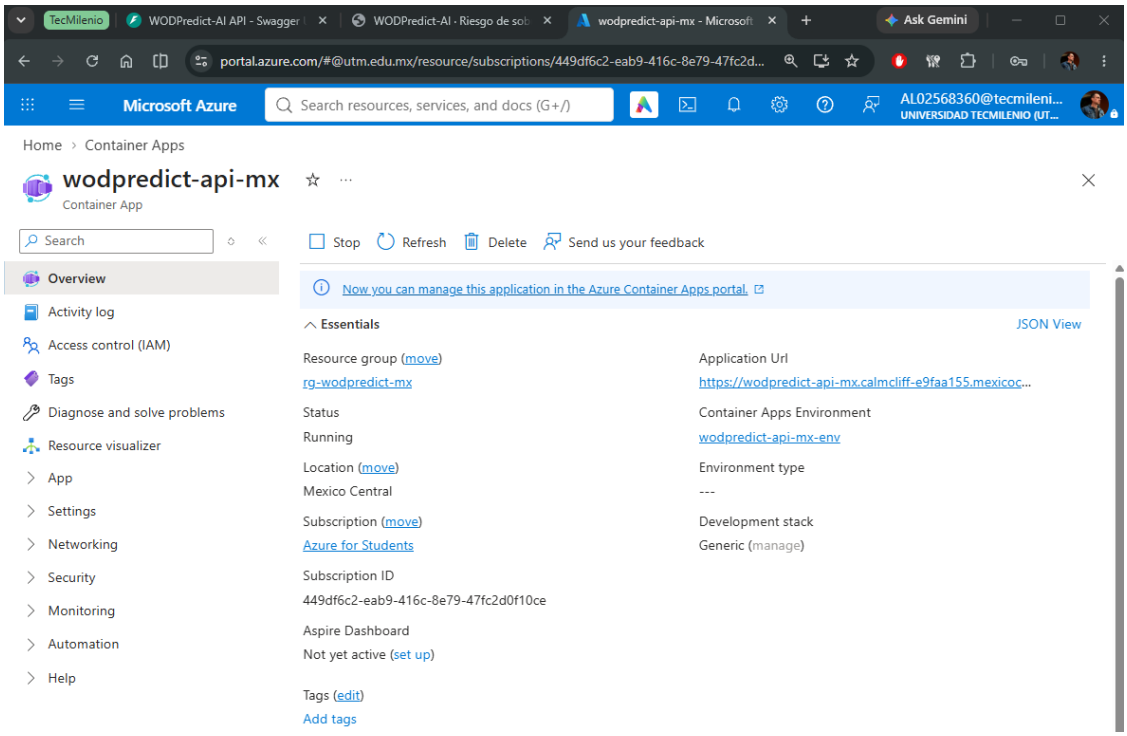


Figura 6. Portal de Azure: Container App wodpredict-api-mx en ejecución (Running) en México Central con la suscripción Azure for Students.

Fase 4. Validación postdespliegue

Paso 9. Validar en la nube. Obtener la URL pública y repetir las mismas pruebas contra Azure:

```
az containerapp show -n wodpredict-api-mx -g rg-wodpredict-mx `
  --query properties.configuration.ingress.fqdn -o tsv
# Abrir https://<fqdn>/app  y  https://<fqdn>/docs
python src/simula_consumo.py https://<fqdn>
$env:API_URL = "https://<fqdn>"; pytest tests/ -v
```

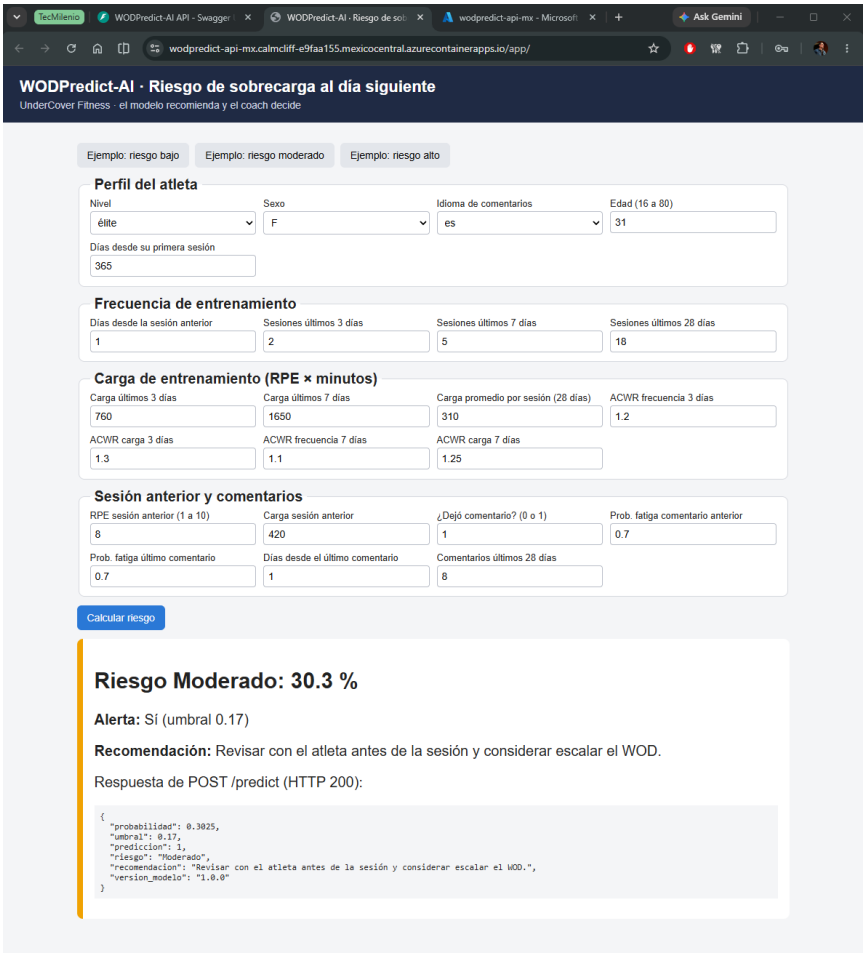


Figura 7. Formulario /app servido desde Azure (México Central): POST /predict responde HTTP 200 con la misma probabilidad que en local (0.3025, riesgo Moderado).

```
(wodpredict) PS C:\Users\ricar\OneDrive - Universidad Tecmilenio\2026 Septiembre-Diciembre\Gestión de proyectos de intel
igencia artificial\Semana 8\WODPredict-AI-FaseII> python src/simula_consumo.py https://wodpredict-api-mx.calmcliff-e9faa
155.mexicocentral.azurecontainerapps.io
GET / -> HTTP 200: {'mensaje': 'API funcionando', 'modelo': 'WODPredict-AI - sobrecarga al día siguiente', 'version': '1
.0.0'}

Atleta de ejemplo 'bajo' -> HTTP 200
  Riesgo Bajo (3.0%) | alerta: 0
  Recomendación: Entrenar según lo planeado.

Atleta de ejemplo 'moderado' -> HTTP 200
  Riesgo Moderado (30.2%) | alerta: 1
  Recomendación: Revisar con el atleta antes de la sesión y considerar escalar el WOD.

Atleta de ejemplo 'alto' -> HTTP 200
  Riesgo Alto (75.0%) | alerta: 1
  Recomendación: Bajar la carga o programar descanso activo; revisar con el atleta.

Dato inválido (prev_rpe = 11) -> HTTP 422: prev_rpe - Input should be less than or equal to 10
(wodpredict) PS C:\Users\ricar\OneDrive - Universidad Tecmilenio\2026 Septiembre-Diciembre\Gestión de proyectos de intel
igencia artificial\Semana 8\WODPredict-AI-FaseII>
```

Figura 8. Cliente externo (simula_consumo.py) contra la URL de México Central: mismos resultados que en local (3.0 %, 30.2 % y 75.0 %) y HTTP 422 para el dato inválido.

Criterio de aceptación: simula_consumo.py devuelve los mismos tres resultados que en local (Bajo 3.0 %, Moderado 30.2 %, Alto 75.0 %) y las pruebas funcionales, de casos extremos, de validación y de consistencia pasan contra la URL pública. **Resultado:** se cumplió; pytest reportó 22 de 23 pruebas aprobadas. Solo L01 (latencia) no alcanza la meta definida para el servicio local; el análisis está en el documento de validación y pruebas (sección 10).

Paso 10. Control de costos y baja del servicio. Limitar a una réplica que se apaga sin uso y, al terminar la evaluación, eliminar todos los recursos:

```
az containerapp update -n wodpredict-api-mx -g rg-wodpredict-mx --min-replicas 0 --max-replicas 1
az group delete --name rg-wodpredict-mx --yes --no-wait
```

9. Operación, monitoreo y mantenimiento

9.1 Monitoreo

Aspecto	Indicador	Cómo se mide	Meta / señal de alerta
Performance del modelo	Recall y precisión semanales	Alertas contra sobrecargas reportadas por el coach	Recall ≥ 0.70 y precisión ≥ 0.35
Performance del modelo	Distribución de predicciones	% de alertas por semana	≈ 24 % en validación; cambio > 10 puntos sugieren data drift
Calidad de entrada de datos	Peticiones rechazadas (HTTP 422)	Logs de la API por campo con error	> 5 % de rechazos indica problemas de captura
Calidad de entrada de datos	Valores faltantes imputados	% de campos opcionales en null	Aumento sostenido = revisar la captura en el box
Latencia y disponibilidad	Tiempo de POST /predict	Pruebas L01 y métricas de Azure Monitor	Local: mediana < 100 ms y p95 < 300 ms; nube: menos de 2 s por consulta
Latencia y disponibilidad	Salud del servicio	HEALTHCHECK sobre GET / cada 30 s; estado de la réplica	Healthy; 3 fallos seguidos = reinicio automático

Logs: en local docker logs wodpredict; en Azure az containerapp logs show -n wodpredict-api-mx -g rg-wodpredict-mx --follow o Log Analytics.

9.2 Mantenimiento

- **Procedimientos correctivos:** ante un error, revisar docker logs o los logs de Azure, corregir, ejecutar pytest tests/ -v y redesplegar; si la nueva versión falla, regresar a la revisión anterior con az containerapp revision activate.
- **Procedimientos preventivos:** revisión semanal de métricas durante el piloto, reentrenamiento mensual con src/entrenar_modelo.py (nueva versión en metadata.json e imagen wodpredict-api:1.1) y actualización controlada de dependencias con versiones fijas.
- **Gestión de incidentes:** 1) detectar (health check o reporte del coach), 2) registrar en un issue de GitHub con fecha, síntoma y versión, 3) mitigar (reiniciar la réplica o volver a la versión anterior), 4) corregir y probar, 5) documentar la causa en el historial de cambios (sección 12).

10. Riesgos, limitaciones y supuestos

Los riesgos se identifican siguiendo la guía de ISO/IEC 23894 (gestión de riesgos de IA).

Tipo	Riesgo o limitación	Impacto	Mitigación
Limitación técnica	Datos sintéticos: el desempeño real puede ser distinto	Alto	Piloto con datos de Wodify (noviembre 2026) y monitoreo semanal del recall
Riesgo de degradación	Data drift: cambio en la programación de WODs o en la población de atletas	Alto	Monitoreo de la distribución de alertas y criterios de reemplazo (sección 7.2)
Riesgo de degradación	Dependencia de la calidad de la captura (RPE, minutos, comentarios)	Medio	Validación de rangos en la API e imputación de faltantes
Limitación técnica	API sin autenticación	Alto con datos reales	Agregar API key o Microsoft Entra ID antes de usar datos personales
Limitación técnica	Falsas alarmas (≈ 24 % de sesiones con alerta)	Medio	El coach revisa cada alerta; niveles de riesgo para priorizar
Limitación técnica	Incompatibilidad de versiones al cargar el model.pkl	Medio	Versiones fijas en requirements.txt y metadata.json
Limitación técnica	Arranque en frío y costo en la nube	Bajo	Escala a 0, máximo 1 réplica y eliminar el grupo de recursos al terminar

Limitación técnica	Restricciones de Azure for Students: MFA obligatoria, solo 5 regiones permitidas y ACR Tasks bloqueado	Medio	Login con --claims-challenge, región México Central y publicación de la imagen local en ACR con docker push (paso 8)
Limitación técnica	Latencia en la nube de ≈ 1.1 s por consulta (17 ms en local) por la conexión HTTPS nueva en cada petición y el ambiente de bajo consumo	Bajo	Aceptable para consultar antes de la sesión; reutilizar conexiones (requests.Session) y mantener una réplica activa si se requiere menor tiempo

Supuestos operativos: el coach captura RPE y minutos al terminar cada sesión; las variables de comentarios (prob_fatiga_*) llegan ya calculadas, igual que en el dataset de entrenamiento; el sistema es una herramienta de apoyo y requiere supervisión humana constante.

11. Consideraciones éticas y uso responsable

- **Riesgos de sesgo:** el recall es menor en atletas élite (0.62) porque en ellos la sobrecarga es menos frecuente; se monitorea por nivel y sexo para evitar que un grupo quede desatendido.
- **Impacto en usuarios:** una falsa alarma solo cuesta una revisión del coach y, como mucho, una sesión más ligera; una sobrecarga no detectada puede terminar en lesión, por eso se prioriza el recall. El atleta puede conocer por qué se ajustó su entrenamiento (carga reciente, ACWR, RPE).
- **Restricciones de uso:** uso exclusivo para prevenir lesiones y ajustar la carga; no se usa para seleccionar, excluir ni evaluar el rendimiento de atletas, ni con fines comerciales o de aseguradoras.
- **Supervisión humana:** el sistema **no toma decisiones finales**; recomienda y el coach decide. Ninguna alerta cambia el entrenamiento sin revisión del coach.
- **Privacidad:** la API no guarda datos ni recibe identificadores; con datos reales se aplicará la ley de protección de datos personales.

12. Trazabilidad y versionado

12.1 Versiones del modelo y del sistema

Elemento	Versión	Dónde se registra
Modelo ML	1.0.0	model/metadata.json y GET /modelo
API	1.0.0	src/app.py (FastAPI) y /openapi.json
Imagen Docker	wodpredict-api:1.0	Docker Desktop / Azure Container Registry
Datos	wodpredict-synth@v1.0	Hugging Face y tag v1.0 de la Etapa 2
Código	v1.0-fase2	Tag de Git en GitHub
Documento	1.0	Este documento

12.2 Historial de cambios relevantes

Versión	Fecha	Cambio
Datos v1.0	Octubre 2026	Dataset sintético publicado y datos limpios versionados (Etapa 2)
Selección de modelo	Octubre 2026	Regresión logística seleccionada con Grid Search y MLflow; umbral 0.5 (Etapa 2)
Modelo 1.0.0	Octubre 2026	Umbral ajustado a 0.17 para cumplir recall ≥ 0.70 (Etapa 3)
API 1.0.0 / imagen 1.0	Noviembre 2026	API REST, formulario web, contenedor Docker, pruebas y despliegue en Azure Container Apps, México Central (Fase II)

12.3 Uso de herramientas de documentación (IA generativa)

Este manual se elaboró con apoyo de una herramienta de IA generativa. Se le indicó el tipo de proyecto, la norma ISO/IEC 23053 como marco y la plantilla de documentación técnica del curso. La IA propuso la estructura de las secciones, la redacción inicial y las listas de verificación. Las decisiones técnicas, los comandos, las métricas y las evidencias provienen de la ejecución real del proyecto (notebook, pytest y despliegue en Azure) y fueron verificadas por el autor contra el repositorio.

13. Glosario de términos

Término	Definición
Sistema de IA	Sistema que genera salidas como predicciones o recomendaciones para objetivos definidos por humanos (ISO/IEC 22989)
Aprendizaje automático (ML)	Proceso que optimiza los parámetros de un modelo con datos para que su comportamiento refleje esos datos
Modelo ML	Construcción matemática que genera una inferencia o predicción a partir de datos de entrada
Datos de entrenamiento	Datos usados para entrenar el modelo (sesiones sintéticas v1.0)
Datos de producción	Datos que recibe el sistema en operación (peticiones a /predict)
Inferencia	Generación de una predicción con un modelo ya entrenado
Verificación y validación	Comprobar que el sistema cumple sus requisitos y su propósito (pruebas y métricas)
API REST	Interfaz que comunica aplicaciones por HTTP con mensajes JSON
Contenedor	Entorno aislado que ejecuta la aplicación con sus dependencias
PaaS	Plataforma como servicio: el proveedor administra la infraestructura y el usuario la aplicación
ACWR	Acute:Chronic Workload Ratio: carga reciente entre carga habitual
RPE	Esfuerzo percibido de la sesión, de 1 a 10
Recall	De las sobrecargas reales, proporción que el modelo detecta
Data drift	Cambio en la distribución de los datos que puede degradar el modelo

14. Anexos técnicos

Anexo A. Diagrama de arquitectura

Ver la Figura 1 (sección 5.1). Flujo: ① el cliente envía el JSON a POST /predict; ② la API valida tipo y rango y pasa una fila al modelo (o responde 422 si un dato es inválido); ③ el modelo devuelve la probabilidad; ④ la API responde en JSON con probabilidad, alerta, riesgo, recomendación y versión. Se ejecuta en un contenedor Docker en local (docker run -p 8000:8000) y en Azure Container Apps (az containerapp up).

Anexo B. Especificación de la API

Método	Endpoint	Entrada	Salida
GET	/	—	{"mensaje": "API funcionando", ...} (health check)
GET	/modelo	—	Ficha técnica del modelo (metadata.json)
POST	/predict	JSON con 23 variables del atleta	probabilidad, umbral, predicción, riesgo, recomendación, version_modelo
GET	/docs	—	Swagger: documentación interactiva
GET	/app	—	Formulario web que consume /predict

```
POST /predict  (Content-Type: application/json)
{"level": "élite", "sex": "F", "language": "es", "age": 31, ..., "notes_last_28d": 8}

HTTP 200
{"probabilidad": 0.3025, "umbral": 0.17, "predicción": 1, "riesgo": "Moderado",
 "recomendación": "Revisar con el atleta antes de la sesión y considerar escalar el WOD.",
 "version_modelo": "1.0.0"}

HTTP 422 (prev_rpe = 11)
{"detail": [{"loc": ["body", "prev_rpe"], "msg": "Input should be less than or equal to 10"}]}
```

El contrato completo está en docs/openapi.json del repositorio.

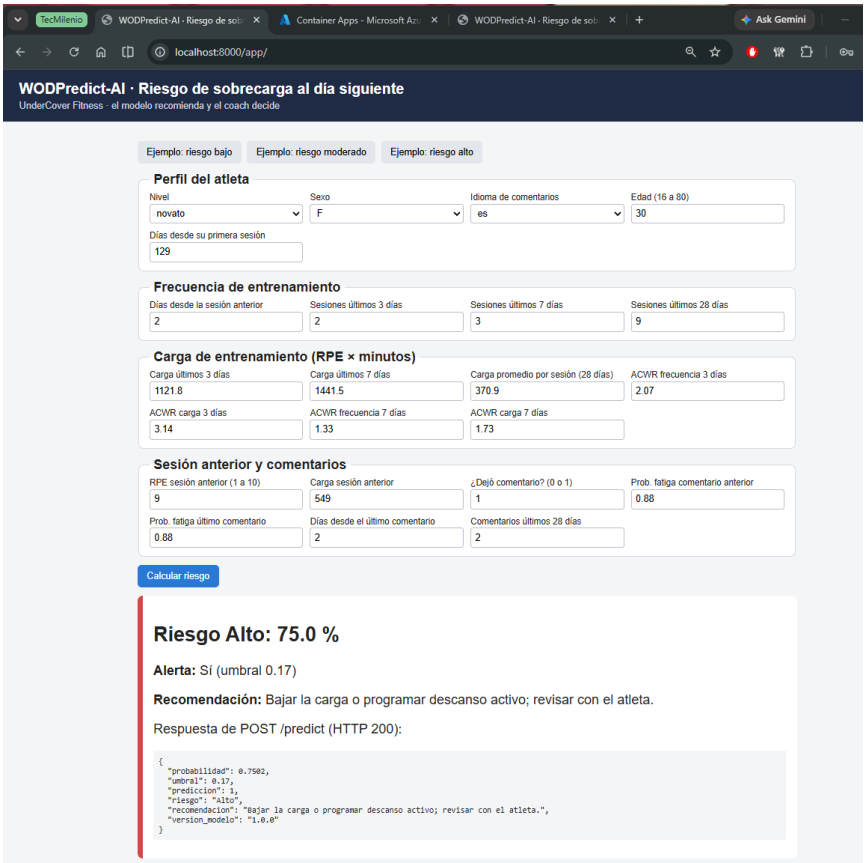


Figura 9. Formulario /app consumiendo POST /predict dentro del contenedor (perfil de riesgo alto).

Anexo C. Configuraciones

Configuración	Valor	Archivo
Imagen base	python:3.12-slim	Dockerfile
Puerto	8000 (variable PORT), igual en local y en Azure	Dockerfile
Health check	GET / cada 30 s, 3 reintentos	Dockerfile
Usuario del contenedor	appuser (sin privilegios)	Dockerfile
Dependencias	Versiones fijas	requirements.txt, requirements-dev.txt
Archivos excluidos de la imagen	notebooks, pruebas, documentos, .venv	.dockerignore
Archivos excluidos del repositorio	.venv, cachés, credenciales (.env), datos descargados, evidencias/ y .docx	.gitignore
Umbral de decisión	0.17	model/metadata.json
Azure Container Apps	Ingress externo, target port 8000, réplicas 0 a 1, región Mexicocentral, imagen wodpredictal02568360.azurecr.io/wodpredict-api:1.0	Comando del paso 8

Anexo D. Scripts (referenciados)

Script	Función
Dockerfile	Receta de la imagen de la API
src/entrenar_modelo.py	Entrena el modelo final y guarda el artefacto y su metadata
src/app.py	API FastAPI: endpoints, contrato de entrada y salida, inferencia
src/model_loader.py	Ubicación del modelo y carga del artefacto y su metadata (lo usan la API y el notebook)
src/simula_consumo.py	Cliente externo que consume la API
frontend/index.html	Formulario web que consume POST /predict
tests/test_api.py	23 pruebas automatizadas contra el contenedor o la URL de Azure
tests/casos_extremos.md	Tabla de casos extremos y de validación: entrada, esperado, real y conclusión
notebooks/fase2_ejecucion.ipynb	Ejecución paso a paso con evidencias
docs/openapi.json	Especificación completa de la API (OpenAPI)